NWIS-Sentinel — 4 Linked Antigravity Build Prompts

Sequential handoff pipeline: Prompt 1 → Prompt 2 → Prompt 3 → Prompt 4, each consuming the previous one's exact output

0. How This Maps to the Official PS Requirements

Before the prompts, here's the coverage map — keep this next to you, it's also your feasibility-slide evidence that nothing was skipped:

Official requirement	Covered by
(i) AI/NLP/OCR/data analytics to extract & structure historical reports	**Prompt 1**
(ii) Interactive map-based visualization, user-defined radius	**Prompt 2**
(iii) Searchable knowledge repository (events, lessons learned, mitigations)	**Prompt 2**
(iv) Correlate geological/drilling/reservoir data by depth & formation	**Prompt 2**
(v) Predictive risk models (mud loss, stuck pipe, overpressure, torque spikes, cementing)	**Prompt 3**
(vi) Real-time alerts & proactive recommendations	**Prompt 3 + Prompt 4**
(vii) User-friendly dashboard for field/office personnel	**Prompt 4**
Data sources (WCRs, DDRs, mud logs, historical parameters, reservoir/geological data, eRTMAC streams, trajectory/survey, casing/cementing/mud program, historical NPT events)	Mapped explicitly to real/synthetic sources in **Prompt 1**, listed in Section 1 below

1. The Dataset Decision (I picked this so your team doesn't have to argue about it)

You asked me to decide — here it is, with reasoning, and it's built to be genuinely **large**, not a toy set:

OIL's listed data source	What we substitute	Why
Well Completion Reports, Daily Drilling Reports	**Synthetic DDR/WCR corpus, LLM-generated at scale (200–400 synthetic reports across 40–60 synthetic wells)**, stylistically grounded in real SPE/OnePetro incident case-study language, covering all 5 named hazards	Real WCRs/DDRs are proprietary; a *large*, stylistically realistic synthetic corpus is the honest, defensible substitute — and generating it at scale via LLM is fast
Drilling/mud logging databases, historical well parameters, trajectory & survey data, real-time (eRTMAC-equivalent) streams	**Volve Field Dataset (Equinor)** — real, public, real North Sea field (operated 2008–2016), released 2018 specifically for research/training use. Contains real well trajectories, real drilling parameters, and real-time drilling data originally in **WITSML format** (the same standard eRTMAC is built on), already parsed to clean CSVs by a university researcher	This is real, not synthetic — genuine drilling physics, not invented numbers
Reservoir & geological data (formation correlation across many wells)	**FORCE 2020 Lithology Prediction Dataset** — a well-known public Norwegian Continental Shelf dataset covering roughly 100+ real wells' well logs with labelled lithology/formation data	This is what makes your dataset genuinely **large** — Volve alone is only a handful of wellbores; FORCE 2020 gives you a hundred-plus real wells for the geospatial/formation-correlation and offset-well similarity demo to look substantial, not thin
Historical NPT event records (mud loss, kicks, stuck pipe, fishing, etc.)	**Real events mined directly out of Volve's own daily reports/well history** wherever documented, supplemented by the synthetic DDR corpus for hazard types Volve doesn't happen to document	Prompt 1 is explicitly instructed to search for and flag any genuine recorded NPT event in Volve's own materials — if found, that becomes your single most powerful demo asset (see Section 2, the "time-travel" backtest)

Total resulting scale: ~100+ real wells (FORCE 2020) + ~5–9 real wellbores with full real-time telemetry (Volve) + 40–60 synthetic wells with rich text narratives = a dataset that is large, majority-real, and fully documented as to which parts are real vs. synthetic (required by the transparency rule below).

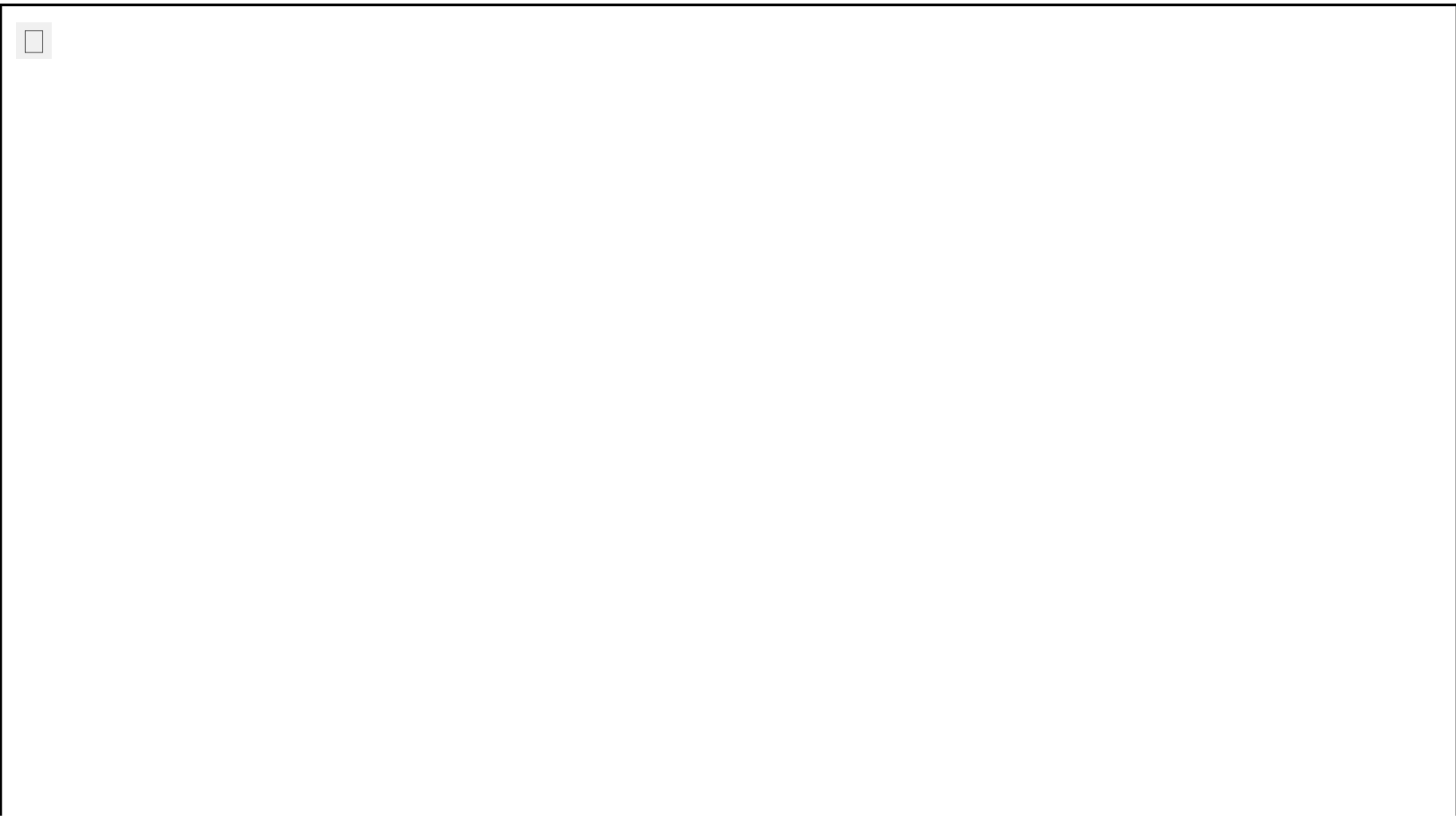
2. The "Mind-Blowing" Centerpiece: The Historical Time-Travel Backtest

This is the single highest-impact addition to your whole project, and it directly answers your request to "prove it would have helped." Here's exactly what it means, so you can explain it to your team before they start prompting:

Take a **real historical well** (from Volve) that has a **documented, known drilling incident** (mud loss, stuck pipe, etc. — Prompt 1's job is to find one, or construct the closest honest analogue if Volve's specific wells don't document one clearly). Feed that well's data into NWIS-Sentinel **exactly as if it were happening live, in chronological/depth order, with the system only ever seeing data up to "now"** — never letting it peek at the future. Record the exact depth/time at which NWIS-Sentinel would have raised an alert. Compare that to the depth/time the incident actually occurred in the real historical record. **The gap between those two points, in metres or hours, is your single most powerful, quantifiable, judge-proof result** — "our system would have given engineers a 45-minute / 60-metre head start on this real historical incident."

This is what Prompt 3 builds, and Prompt 4 displays prominently on the dashboard as the flagship demo.

3. Rules That Apply to ALL FOUR Prompts (paste this block at the top of every single prompt you give Antigravity)



GLOBAL RULES FOR THIS PROJECT (apply to every module you build):

1. NO HARDCODING, NO FABRICATED OUTPUTS. Every number, prediction, or result must come from real computation on real or clearly-labelled-synthetic data. Never simulate a "successful" result by hardcoding an expected output.

2. NO SILENT API STUBS. If a task requires an external API or service (OCR, geocoding, an LLM, embeddings, cloud storage), and no credential is available in the environment, STOP and explicitly tell me exactly which API/credential you need and why. Do not silently mock it, fake a response, or skip the step.
Available to you: Google Cloud (Vision API for OCR, Vertex AI / Gemini for LLM tasks, Maps Platform for geospatial), Hugging Face (Inference API and open model downloads).
Ask me for the specific key/credential/project-ID you need, by name.

3. FULL TRANSPARENCY, NO BLACK BOXES. Every prediction, similarity score, or risk alert must be accompanied by: (a) the exact feature values that produced it, (b) the weights or model parameters applied, (c) a plain-language explanation of why this output was produced. If you use any model that isn't inherently interpretable (e.g. a neural network or gradient-boosted tree), you must also output feature importances or a SHAP-style explanation alongside every prediction. A user should never see a number with no visible reasoning behind it.

4. DOCUMENT YOUR WORK. When you finish, create a file named README_[module_name].md containing:
- What this module does, in plain language
- Exact tech stack used (languages, libraries, models, APIs) and why each was chosen
- Exact input file(s)/schema(s) it expects
- Exact output file(s)/schema(s) it produces
- Which parts of your output are derived from REAL data vs SYNTHETIC/generated data — be explicit and honest about this distinction everywhere it applies
- Known limitations and what you'd need to improve this with more time/real OIL data

5. DATASET SCALE. Do not settle for a tiny toy dataset. Follow the exact dataset sourcing instructions given in your specific prompt below — the target is dozens to 100+ wells, not 3–5.

6. USE REAL, WORKING CODE. Every module must actually run end-to-end on the data you build/fetch, not just describe what it would do.

4. PROMPT 1 — Data Foundation, OCR & NLP Document Intelligence

(Give this to Antigravity first. Person 1 owns this workspace.)

[PASTE THE GLOBAL RULES BLOCK FROM SECTION 3 HERE]

ROLE: You are building Module 1 of a 4-module drilling-intelligence system called NWIS-Sentinel, for Smart India Hackathon PS SIH26121 (Oil India Limited — Nearby Wells Intelligence System). You own ONLY the data foundation, OCR, and NLP extraction layer. Do not build anything from Modules 2-4 (geospatial, prediction, dashboard) — just produce clean, well-documented output that those modules will consume.

OBJECTIVE:
Build a large, real-and-honestly-labelled dataset of historical wells, and extract structured, numeric-comparable knowledge from all available text sources using OCR + NLP.

STEP 1 — ACQUIRE REAL DATA (do this yourself; these are public datasets):
a) Fetch the Volve Field Dataset real-time drilling data (originally WITSML, already parsed to CSV by University of Stavanger researcher Alexander Tunkiel, publicly hosted). Pull the real-time drilling parameter CSVs (torque, RPM, flow, standpipe pressure, ROP, depth) for as many of Volve's wellbores as are available. Also pull well trajectory/survey data and formation top data from Equinor's Volve data release if accessible.
b) Fetch the FORCE 2020 Lithology Prediction Dataset (public, ~100+ Norwegian Continental Shelf wells with labelled well logs / formation data). Use this to build a LARGE multi-well formation/geological reference set.
c) IMPORTANT: search specifically through whatever daily-report / well-history documentation is available for the Volve wells for any GENUINE documented drilling incident (mud loss, stuck pipe, kick, NPT event, etc.). If you find one, flag it clearly and prominently in your README — this becomes the seed for Module 3's historical backtest demo. If nothing is clearly documented, say so honestly rather than inventing one.
d) If any of these public datasets require a download you cannot complete automatically (size, access restrictions), tell me exactly what you need me to manually download and place in the project folder, with the exact URL.

STEP 2 — GENERATE A LARGE SYNTHETIC REPORT CORPUS (to stand in for proprietary WCRs/DDRs):
Generate 200-400 synthetic Daily Drilling Report / Well Completion Report style text snippets, across 40-60 distinct synthetic wells, using an LLM (use Google Vertex AI/Gemini or Hugging Face — ask me for the credential if you don't have one configured). These must:
- Cover all 5 named hazard types: mud loss, stuck pipe, overpressure zones, torque spikes, cementing issues.
- Be stylistically grounded in real SPE/industry incident-report language patterns (paraphrase style/structure, do not copy real copyrighted text verbatim).
- Include realistic depth references, formation names, and equipment mentions consistent with the real Volve/FORCE 2020 formation and depth ranges, so later modules can compare synthetic and real data on the same numeric scale.

- Be clearly tagged as "synthetic" in your data (a boolean/source field), never mixed silently with real data.

STEP 3 — OCR PIPELINE:
Render at least 15-20 of the synthetic reports as scanned-looking document images (PDF or image format), then build a real OCR pipeline (use Google Cloud Vision API — ask me for the credential — or Tesseract as a free fallback if no credential is available) to extract text back out of these images. This demonstrates the PS's explicit OCR requirement. Report OCR accuracy (character/word error rate against the known original text) transparently.

STEP 4 — NLP EXTRACTION PIPELINE:
For ALL text (real report-style text + synthetic + OCR-extracted), build:
a) Entity extraction: identify Formation, Depth, Equipment, EventType, Intervention, Outcome spans. Start with rule-based/pattern matching (spaCy) for depths/units, and use an LLM-based few-shot extraction OR a fine-tuned transformer (Hugging Face) for the rest. State clearly in your README which approach you used and why.
b) Causal/relation extraction: link events to interventions to outcomes using dependency parsing + lexical trigger patterns ("as a result of", "following", "due to").
c) Event-type normalization: map varied phrasings to a FIXED, versioned vocabulary of canonical event-type IDs (e.g. EVT_TORQUE_UP, EVT_MUD_LOSS_PARTIAL, EVT_STUCK_PIPE, EVT_CEMENTING_FAILURE, EVT_OVERPRESSURE_DETECTED, EVT_LCM_APPLIED, EVT_CIRC_RESTORED,
etc. — cover all 5 hazards' relevant event states). Use sentence-embedding similarity (Hugging Face sentence-transformers) against reference phrases for this mapping. Save this vocabulary as its own versioned file — this is the most important shared artifact in the whole project, and Modules 2-4 depend on it being stable.

STEP 5 — PRODUCE THE EXACT OUTPUT FILES (Module 2 will consume these directly):
1. `wells_metadata.json` — one record per well: well_id, source ("real_volve" / "real_force2020" / "synthetic"), latitude, longitude (approximate/synthetic if not available for a well, clearly flagged), trajectory points, formation tops, mud program, casing design, BHA type.
2. `events.jsonl` — one JSON object per extracted event: well_id, event_id, depth_m, timestamp, event_type_id (from the canonical vocabulary), formation_id, source ("report"/"ocr"/"synthetic"), source_snippet_id, confidence, raw_text.
3. `telemetry/<well_id>.csv` — real numeric time-series per well (torque, rop, flow, pressure, depth, timestamp) for every Volve well you were able to pull.
4. `event_type_vocabulary.json` — the full canonical event-type list, versioned.
5. `README_module1_data_and_nlp.md` — per the global documentation rule in Section 3, PLUS an explicit table of exactly how many real vs synthetic wells/events/reports you produced, and the flagged historical incident (if found) from Step 1c.

Do not proceed to build anything beyond this. Confirm all 5 output files exist and are valid before finishing.

5. PROMPT 2 — Geospatial Visualization, Offset-Well Correlation & Knowledge Repository

(Give this to Antigravity after Prompt 1 finishes. Person 2 owns this workspace. Feed it Prompt 1's exact output files.)

[PASTE THE GLOBAL RULES BLOCK FROM SECTION 3 HERE]

ROLE: You are building Module 2 of NWIS-Sentinel. You consume Module 1's output exactly as produced — do not regenerate or re-extract data; if a required field is missing from Module 1's files, tell me rather than inventing it.

INPUT (from Module 1, already built — read these files):
- wells_metadata.json, events.jsonl, telemetry/*.csv, event_type_vocabulary.json
(exact schemas are documented in README_module1_data_and_nlp.md — read that file first)

OBJECTIVE:
Build three things: (a) an interactive geospatial map with radius-based nearby-well search, (b) a hazard-specific offset-well similarity engine, (c) a searchable knowledge repository.

STEP 1 — GEOSPATIAL MAP (PS requirement ii):
Build an interactive map (use Google Maps Platform JavaScript API — ask me for the API key if not already configured; or Leaflet + OpenStreetMap as a free fallback if you prefer not to depend on a paid key) that:
- Plots every well from wells_metadata.json at its lat/long.
- Lets a user select any well and a radius (in km), and highlights all other wells within that radius.
- Color-codes wells by data source (real Volve / real FORCE2020 / synthetic) so it's always visually obvious which pins are real vs synthetic, per the transparency rule.

STEP 2 — HAZARD-SPECIFIC OFFSET-WELL SIMILARITY ENGINE (this is the project's core novelty — implement it exactly as specified, do not substitute a generic single similarity score):
For each of the 5 hazard types (mud loss, stuck pipe, overpressure, torque spike, cementing issue), define a DIFFERENT weighted feature set and compute:

Similarity(well_i, hazard_h) = sum over k of [w_(h,k) * sim_k(well_i, target_well)]

Where sim_k are individually computed, NORMALIZED (0-1) similarity functions:
- Categorical features (formation type, BHA type, mud type): Jaccard similarity.
- Continuous numeric features (mud weight, pore pressure estimate): 1 minus normalized Euclidean distance, or a Gaussian kernel similarity.
- Trajectory shape (inclination/azimuth vs depth curves): Dynamic Time Warping distance, converted to similarity (use the `fastdtw` or `dtaidistance` Python library — do not hand-roll DTW unless those libraries are unavailable).

For the weights w_(h,k): use the Analytic Hierarchy Process (AHP) — construct a

pairwise-comparison matrix per hazard type based on documented drilling-engineering reasoning (e.g., cite that formation/pore-pressure dominate mud-loss risk, trajectory/BHA dominate stuck-pipe risk), derive weights from the matrix's principal eigenvector. Store the exact pairwise matrices and resulting weights in a human-readable file so this is fully auditable, per the no-black-box rule.

STEP 3 — GEOLOGICAL/FORMATION CORRELATION (PS requirement iv):
Using formation-top data across ALL wells (especially the FORCE 2020 wells, since you have 100+ of them), build a correlation view: for a given depth range and formation, show which other wells encountered the same formation, at what depth, and with what logged characteristics. This should be queryable by depth AND by formation name.

STEP 4 — SEARCHABLE KNOWLEDGE REPOSITORY (PS requirement iii):
Build a search interface over events.jsonl (and the underlying raw_text) that supports:
- Keyword search (simple full-text search is fine — use a lightweight library like `whoosh`, or Elasticsearch if you want to demonstrate more infrastructure).
- Semantic search using sentence-transformer embeddings (Hugging Face) for "find events similar in meaning to this description," not just exact keyword matches.
- Filters by hazard type, well, formation, and date.

STEP 5 — PRODUCE THE EXACT OUTPUT (Module 3 and Module 4 will consume these):
1. `analog_wells.json` — for every well and every hazard type, the ranked list of analog wells with their per-feature similarity breakdown AND the final weighted score.
2. `ahp_weights.json` — the full pairwise-comparison matrices and derived weights per hazard, for full auditability.
3. `formation_correlation.json` — the cross-well formation/depth correlation table.
4. A working map application (note the exact run command/URL in your README).
5. A working search interface (note the exact run command/URL in your README).
6. `README_module2_geospatial_and_similarity.md` per the global documentation rule.

Do not build the predictive risk models or the dashboard — that is Modules 3 and 4.

6. PROMPT 3 — Predictive Risk Analytics & the Historical Time-Travel Backtest

(Give this to Antigravity after Prompt 2 finishes. Person 3 owns this workspace. Feed it Prompt 1 AND Prompt 2's outputs.)

□

[PASTE THE GLOBAL RULES BLOCK FROM SECTION 3 HERE]

ROLE: You are building Module 3 of NWIS-Sentinel. You consume Module 1's and Module 2's outputs exactly as produced.

INPUT (already built — read these first):
- From Module 1: events.jsonl, telemetry/*.csv, event_type_vocabulary.json
- From Module 2: analog_wells.json, ahp_weights.json
(schemas documented in README_module1_*.md and README_module2_*.md — read those first)

OBJECTIVE:
Build (a) predictive risk models for all 5 hazards, (b) a live-telemetry simulator, (c) an event-sequence matching engine, and (d) THE HISTORICAL TIME-TRAVEL BACKTEST — this last piece is the single most important deliverable in the whole project, treat it as such.

STEP 1 — LIVE TELEMETRY SIMULATOR:
Build a service that replays a REAL Volve well's telemetry.csv row-by-row (or a synthetic well's telemetry if no real incident well is suitable), at a controllable speed, over a WebSocket or simple polling endpoint — simulating what a real eRTMAC/WITSML feed would provide. Explicitly label this in code comments and the README as "Live Telemetry Simulator — production version would connect directly to eRTMAC's WITSML feed."

STEP 2 — PRECURSOR/ANOMALY DETECTION (feeds the predictive models):
For each of the 5 hazards, implement precursor detection on the live-replayed telemetry: start with transparent statistical methods (rolling z-score, CUSUM change-point detection) as the primary, fully-explainable baseline. If time allows, add an autoencoder (reconstruction-error based anomaly score) as a secondary signal — but the statistical baseline must always be present and must always be shown alongside any deep-learning output, per the no-black-box rule.

STEP 3 — EVENT-SEQUENCE MATCHING (PS requirement v, the technical core of prediction):
Convert live telemetry into the same event-type-token sequence format used in Module 1's event_type_vocabulary.json. Implement sequence matching against every analog well's historical event sequence (from Module 2's analog_wells.json) using:
- Sequence alignment (Needleman-Wunsch global alignment or Smith-Waterman local alignment — use Biopython's pairwise alignment module, or implement it directly; this is a well-established, explainable dynamic-programming algorithm, not a black box).
When a strong partial match is found, compute risk using a Wilson score confidence interval (use `statsmodels.stats.proportion.proportion_confint` with method='wilson') over the matched analogs' actual outcomes — e.g. "3 of 5 matched analogs experienced this event next (60%, 95% CI: 23%-88%)" — NEVER report a bare percentage without its confidence

interval, this is a core transparency requirement of this project.

STEP 4 — THE HISTORICAL TIME-TRAVEL BACKTEST (the flagship deliverable):
Using the real historical incident well flagged by Module 1 (or the most realistic available real/synthetic candidate if none was clearly documented — be explicit about which case you used and why):
a) Replay that well's telemetry through your entire pipeline (Steps 1-3) IN STRICT CHRONOLOGICAL/DEPTH ORDER, ensuring the system at each point only has access to data up to that point in time — never allow any future data leakage.
b) Record the exact depth/timestamp at which your system's risk score/alert would have first crossed an actionable threshold.
c) Compare this to the ACTUAL depth/timestamp of the real recorded incident.
d) Compute and clearly report the "lead time" your system would have provided (in metres and/or minutes), and produce a simple time-series plot showing: the telemetry curve, the moment of your system's alert, and the moment of the actual incident, side by side.
e) Save this as a standalone, clearly-labelled result — this is your single most important demo asset.

STEP 5 — PRODUCE THE EXACT OUTPUT (Module 4 will consume these):
1. `risk_predictions.jsonl` — live risk scores per hazard, each with full feature/weight breakdown and Wilson confidence interval (no bare numbers).
2. `sequence_matches.json` — matched historical analogs and their alignment scores per live query.
3. `backtest_result.json` + `backtest_plot.png` — the full time-travel backtest result described in Step 4, with the computed lead-time metric front and center.
4. The live telemetry simulator (note exact run command in README).
5. `README_module3_prediction_and_backtest.md` per the global documentation rule, with the backtest lead-time result stated in the first paragraph.

Do not build the dashboard or the LLM briefing layer — that is Module 4.

7. PROMPT 4 — Knowledge Graph, GraphRAG, LLM Briefing & Final Dashboard

(Give this to Antigravity last. Person 4 — you — owns this workspace. Feed it all three prior modules' outputs.)

[PASTE THE GLOBAL RULES BLOCK FROM SECTION 3 HERE]

ROLE: You are building Module 4 of NWIS-Sentinel — the integration layer that ties Modules 1-3 together into one coherent, explainable, demo-ready system.

INPUT (already built — read all of these and their README files first):
- Module 1: wells_metadata.json, events.jsonl, telemetry/*.csv, event_type_vocabulary.json
- Module 2: analog_wells.json, ahp_weights.json, formation_correlation.json, the map app
- Module 3: risk_predictions.jsonl, sequence_matches.json, backtest_result.json, backtest_plot.png, the live telemetry simulator

OBJECTIVE:
Build the Knowledge Graph, a GraphRAG evidence-retrieval layer, an LLM briefing generator that is provably non-hallucinating, and the final unified dashboard (PS requirement vii).

STEP 1 — KNOWLEDGE GRAPH:
Build an in-memory property graph (use `networkx`) with node types: Well, Formation, Event, Hazard, Intervention, Outcome, ReportSnippet. Populate it from Module 1's events.jsonl and Module 2's analog_wells.json. Include edges: DRILLED_THROUGH, HAD_EVENT, FOLLOWED_BY (to represent event sequences), MITIGATED_BY, LED_TO, ANALOG_FOR_HAZARD (from Module 2's output directly — do not recompute similarity here), EXTRACTED_FROM (linking every event back to its exact source report snippet, for full citation traceability).

STEP 2 — GraphRAG RETRIEVAL (do NOT build plain vector-similarity RAG over all documents — that is explicitly the weaker, more common approach; build this specific design instead):
When a query arrives (either automatically from Module 3's live risk output, or typed by an engineer), FIRST restrict the graph to only the subgraph of wells already identified as relevant analogs by Module 2/3 (hazard-specific pre-filtering), THEN do fine-grained retrieval (sentence-transformer embedding similarity) only WITHIN that pre-filtered subgraph. This pre-filter-then-retrieve order is the project's core RAG novelty — do not skip the pre-filter step.

STEP 3 — LLM BRIEFING GENERATOR (use Google Vertex AI/Gemini or another LLM API — ask me for the credential/project ID if not configured):
Generate the engineer-facing briefing text with a FORCED-CITATION prompt template: every factual sentence the LLM produces must include a bracketed reference to a specific graph node ID or report-snippet ID. After generation, run an automatic verification pass: for every cited sentence, confirm the cited source actually contains/supports the claim (a simple keyword/entailment check is sufficient); flag or remove any sentence that fails this check. Log every verification result — this is your core "no hallucination" proof, and it must be visibly demonstrable, not just claimed.

The LLM must NEVER generate the numeric risk score itself (that comes only from Module 3), and must ALWAYS explicitly state when matched analogs disagree (use Module 3's Wilson interval output directly — never let the LLM paraphrase away the uncertainty into false confidence).

STEP 4 — FINAL DASHBOARD (PS requirement vi and vii):
Build one unified, user-friendly dashboard (React, or Streamlit if the team needs speed) that brings everything together:
a) The geospatial map (Module 2) with radius search.
b) The searchable knowledge repository (Module 2).
c) A live view: current well's telemetry streaming in (Module 3's simulator), with real-time risk alerts and recommendations appearing as they're generated, each showing its full feature/weight breakdown (no-black-box rule) and its LLM-generated, citation-verified briefing.
d) A prominently-featured "Historical Time-Travel Backtest" panel showing Module 3's backtest result and lead-time metric — make this visually the centerpiece of the demo, e.g. a depth-synchronized playback showing the alert moment vs. the actual incident moment side by side.
e) A depth-synchronized multi-well playback view comparing the current/replayed well against its matched historical analogs.

STEP 5 — PRODUCE:
1. The full running dashboard application (exact run command in README).
2. `README_module4_integration_and_dashboard.md` per the global documentation rule, PLUS a final end-to-end system summary: total real vs synthetic data used across all 4 modules, the backtest lead-time headline result, and a list of every API/credential actually used in the final system.

This is the final module — once this is done, the project should be fully demoable end-to-end from a single dashboard.

8. What You Need to Provide (vs. What Antigravity Should Self-Source)

You provide, before starting:

Google Cloud project ID + API keys enabled for: Vision API (OCR), Vertex AI/Gemini (LLM), Maps Platform (geospatial) — set these as environment variables/secrets in your Antigravity workspace settings, not pasted into chat.
A Hugging Face access token (for downloading models / using the Inference API).

If Antigravity's environment has restricted/slow internet access for large downloads: manually download the Volve real-time CSV subset (from the University of Stavanger's parsed-file page) and the FORCE 2020 dataset yourself, and place them in a shared project folder/Drive that all 4 workspaces can read from — large multi-GB downloads inside an agent session can be slow or get interrupted, so pre-staging this yourself is the safer bet.
The official SIH26121 PS text (so each workspace's agent has the exact original wording as reference alongside these prompts).

Antigravity/the agent should self-source:

The actual dataset *files* within the subset you've pointed it to (it can fetch from the public URLs itself once it knows exactly which files/subset to pull — don't make it guess the whole 5TB Volve release, point it at the specific parsed real-time CSV page and the FORCE 2020 dataset page).
All code, models (via Hugging Face), and the synthetic report generation.
Its own README documentation, as instructed in every prompt.

9. Execution Order Reminder

Run strictly in this order, each waiting for the previous to fully complete and produce its output files: **Prompt 1 → Prompt 2 → Prompt 3 → Prompt 4.** Do not start Prompt 2 until you've confirmed Prompt 1's 5 output files exist and match the documented schema — a broken handoff here is the most likely single point of failure in a sequential build, more so than in a parallel one, since each step now blocks the next.